Sample Chapter: Core JDO ISBN 0-13-140731-7

The chapters of the book titled "Core JDO" are being made available for the purposes of review to allow the community to participate in the review and editing of the chapters of the book.

To add your comments

- Simply send your uninhibited comments and feedback to the authors at jdobook@yahoogroups.com
- Or click and add a note using Acrobat where needed. E-mail the PDF back to the authors at jdobook@yahoogroups.com
- If you would like to receive the chapters in an alternate format, have any questions, concerns or please contact the authors at jdobook@yahoogr oups.com

CHAPTER 2

"We are made to persist. That's how we find out who we are"- Tobias Wolff In Pharaoh's Army

Object Persistence Fundamentals

This chapter takes a look at what's technically behind object persistence. Different ways to implement persistence frameworks are presented, explained and compared. A broad view is taken at the JDBC-based object-relational mapping solutions and the effort of a implementation from scratch of such a framework is outlined.

At the end of the chapter the reader should have a good understanding of the different persistence principles and design goals of JDO and some of the decisions taken by the JDO specification's expert group.

2.1 PERSISTENCE IN APPLICATIONS

The initial idea of object persistence is to save the current state of an object to some kind of persistent storage, which can be a file, a disk, a flash memory or a network connection. After the system is powered up again, or an application is restarted, it should be possible to re-create the state of an object exactly as it has been before.

The persistence algorithms can either be implemented by the object's class itself or it can be made persistent *magically* by the operating system, for instance. Thanks to object-oriented programming, it is easy to implement the first case in a helper class that can read and write all basic types like int, short, byte, boolean, String, Date and so on. Such a helper class works as an adapter between the storage media and the object contents. If an interface is used for all classes that should become persistent, it is also easy to make a closure of objects persistent, and read the state back into memory. We will drill more deeply into this approach later.

The other way to load and save the object state persistently can be implemented by the runtime system, the operating system or, in case of Java, by the virtual machine. The idea is

that this environment already has all the information about the memory layout of objects, their references to other objects and so on. It just needs to copy a chunk of data to or from disk. This kind of *orthogonal persistence* becomes difficult to realize because the underlying runtime or operating system cannot properly restore semantics without notion of surrounding application environment. Examples of such persistence-incapable types are network connections (sockets), file handles or window positions.

In the past several vendor-neutral groups have tried to solve and propose standards for the *persistence* problem: Project Forest (Sun's Orthogonal Persistence research project), Java Blend, Object Data Management Group (ODMG), EJB CMP and many others. Some of them where either too complex or too far away from real-world applications.

In the next sections a closer look is taken at already existing solutions and how or when they should be used in applications.

2.2 JDK BINARY SERIALIZATION

Serialization is the basic persistence support that is built into the Java language. It is named serialization because the objects are written or read sequentially as a series of bytes. An object is serializable if it's class implements the Serializable interface and all it's non-transient members are as well serializable. When an object is serialized, the default algorithm traverses the closure of the object graph and writes all reachable objects to a stream. If a member of a class is tagged transient, the default algorithm skips this field.

When the stream is read back, the equivalent object graph is rebuilt keeping transient members initialized to their null values. By tagging a field as 'transient', it's referenced object or content will not be serialized by the default algorithm. On the contrary, fields that are not tagged 'transient' are not checked for their serializability by the Java compiler. For example, ArrayList itself is serializable because it implements the Serializable interface, but since an instance of ArrayList might refer any instance of java.lang.Object, the graph might not be serializable.

Transient and persistent classes, fields and instances.

It is essential to persistence frameworks that the capability to persist is a property of declared fields as well as classes. If this capability is also a property of in-memory instances or bare objects, not all instances of persistence-capable classes can be persisted and runtime errors may occur.

2.2.1. The Serialization API

The Serializable interface has no declared methods. It is just a tagging interface, which enables the default algorithm in the Java runtime library. Though there are two methods that

are used frequently, readObject and writeObject, a brief look is taken at the basic concepts and default algorithm of Serialization. The following code copies a BookObject into a byte array and back into another BookObject:

```
import java.io.*;
import java.util.*;

class BookObject implements Serializable
{
      String name;
      BookObject(String n)
      {
         name = n;
      }
}


public class SerialTest
{
  public static void main(String args[])
    throws Exception
    {
      BookObject object = new BookObject("foo");

      // create a buffer to write the object into:
      ByteArrayOutputStream bo =
                new ByteArrayOutputStream();

      // create a stream for serial object data:
      ObjectOutputStream oo = new ObjectOutputStream(bo);

      // write the object (and all its members recursively)
      oo.writeObject(object);

      // don't forget to close the stream:
      oo.close();

      // now lets get the object back from the stream:
      // get a new stream that reads the buffer of the
      // previous stream:
      ByteArrayInputStream bi =
                new ByteArrayInputStream(bo.toByteArray());

      // get a stream that can read objects:
```

```
        ObjectInputStream oi = new ObjectInputStream(bi);

        // cast the read object to our type:
        BookObject back = (BookObject)oi.readObject();

        // never ever forget to close a stream:
        oi.close();
    }
```

Three important things can be seen here:

- ObjectOutputStream and ObjectInputStream are the main working horses for serialization.
- Secondly, a String seems to be serializable and is written to the stream by some magic. There is no code to read or write that member in the BookObject class.
- Lastly, the returned object has to be casted to the destination type. Though the DataInputStream "knows" which object to create from the stream, there must be some generic method that returns simply objects.

In the example code, ByteArray may be replaced by any other Java stream implementation. This is what makes Serialization a powerful component of the Java platform. Simply anything may be *streamed* through a pipe, network connection, file, or into a buffer. Other components make heavy use of Serialization, such as Remote Method Invocation (RMI) for example. Needless to say that Serialization works across platforms and is independent of the CPU byte order or other operating system dependencies.

2.2.2. Versioning and Serialization

One of the obstacles with serialization is version handling. When anything changes in the class layout, the class name, package name, inheritance, fields or access modifiers, a new version of the class must be recognized by the serialization runtime library. This is done by a so-called serial version unique identifier (serial version UID) which is a kind of hash code over all properties of a class. The default algorithm throws a StreamCorruptedException or an InvalidClassException if the serialized object data is incompatible with the class in memory. That is one reason why developers need to overload the readObject and writeObject methods with own implementations. Here is an example how to handle the versioning problem:

```
class Author
    implements Serializable
{
    String name;
```

```
    Author(String n)
    {
        name = n;
    }

    private void writeObject(ObjectOutputStream out)
        throws IOException
    {
        int version = 1;
        out.writeInt(version);
        out.writeUTF(name);
    }

    private void readObject(ObjectInputStream in)
        throws IOException, ClassNotFoundException
    {
        int version = in.readInt();
        if (version == 1) {
            name = in.readUTF();
        } else {
            throw new IOException(
                  "Invalid version: "+version);
        }
    }
}
```

Now that a version is provided with every object in the stream, the readObject method can easily distinguish old and new objects. The second version of the Author class gets its name attribute split into first and last name:

```
class Author implements Serializable
{
    String firstName;
    String lastName;

    Author(String first, String last)
    {
        firstName = first;
        lastName = last;
    }

    private void writeObject(ObjectOutputStream out)
        throws IOException
    {
```

```
        int version = 2;
        out.writeInt(version);
        out.writeUTF(firstName);
        out.writeUTF(lastName);
    }

    private void readObject(ObjectInputStream in)
        throws IOException, ClassNotFoundException
    {
        int version = in.readInt();
        if (version == 2) {
            firstName = in.readUTF();
            lastName = in.readUTF();
        } else if (version == 1) {
            // fallback:
            String name = in.readUTF8();
            firstName = guessFirstName(name);
            lastName = guessLastName(name);
        } else {
            throw new IOException(
               "Invalid version: "+version);
        }
    }
}
```

2.2.3. When Object Serialization should be used

An application should use Serialization when simple data types have to be saved persistently. Configuration data, which is entered through dialog boxes, or window positions in GUI applications are a good candidate for Serialization. When the application version changes, such settings may be simply ignored and complex version handling may not be required. Another use case is RMI: an RMI method may take any object as parameter, if the object and it's referenced attributes are serializable. The same applies to return values. Complex version handling is required only for applications that must support incompatible client and server application versions.

2.2.4. When Object Serialization can not be used

Serialization does however have serious limitations that exclude it as a candidate technology for storing even simple domain objects (e.g. clients, bills, issues, etc.) of real-world applications. Most notably, Java JDK Serialization lacks:

- **Query facility:** Once objects are serialized into a stream, there is no way to ask e.g. the stream to "return all objects which…". The readObject() method will simply return the next object in the stream, with all objects reachable from it.
- **Partial read or update:** While partial reads and updates as well as caching features are "only" performance and memory usage enhancing features of other persistence solutions, the lack of such approaches in Serialization makes it useless for big quantities of data.
- **Life cycle management:** The deliberately simple Serialization API (as seen above) has no notion of an object's "state", i.e. an object being "dirty" etc. Objects exist in memory, and are explicitly written to and read from dumb streams, it really doesn't go further. As mentioned, this is of course perfectly suitable for purposes such as RMI or persisting very simple configuration information, but not for domain data.
- **Concurrency and transactions:** Serialization completely lacks the notion of ACID transactions and is not suitable for applications with concurrent data access. It is, for example, impossible to have two threads write to or read from one and the same stream at the same time.

The next section will examine how relational databases, which were initially built to address the listed issues, can be used with an object persistence framework as data storage technology.

2.2.5. JDO and Object Serialization

JDO has taken two ideas from the Java Serialization framework: The transient keyword and the persistence-by-reachability principle (Though the principle is older than Java, it was first used by Serialization in Java). The transient keyword sets the default for fields to non-persistent. The persistence declaration of fields can be overwritten in a JDO metadata description file, so that fields declared transient in the Java source file can be made persistent, and fields that are not declared transient can be made non-persistent in the JDO metadata file.

Another way to use JDO together with Serialization is to store an object graph into a byte array and then copy that byte array into a persistent field. This technique is useful for classes that are serializable, but not persistence-capable. An optional feature of JDO is to automatically support serializable fields, if they are not persistence-capable.

2.3 Object-Relational Mapping

Object-Relational (O/R) mapping is a technique to map between existing or newly developed domain object models and relational database models respectively. Java relations

between objects, classes and their fields have to be mapped somehow to database relations, tables and columns in a relational database.

The domain model, mentioned in Chapter ###, is often given up for technical reasons like performance or space requirements when the application matures. In this case, object-relational mapping is also a technique to re-engineer complex, existing database models. The object-oriented "is-a" and "has-a" relationships can only be poorly mapped to relational databases, where a record or related information often spans a number of tables and has to be assembled by join operations.

While it is often perfectly possible to work with the JDO API using an O/R-based JDO implementation without understanding the details of the underlying mapping issues, it can be helpful to understand them. The rest of this chapter can be read to understand the background of questions that you would generally trust your JDO vendor has solved for you. Alternatively, the remainder of this chapter could be skipped altogether at this point, to plunge straight away into the usage of JDO in the following chapters.

2.3.1. Classes and Tables

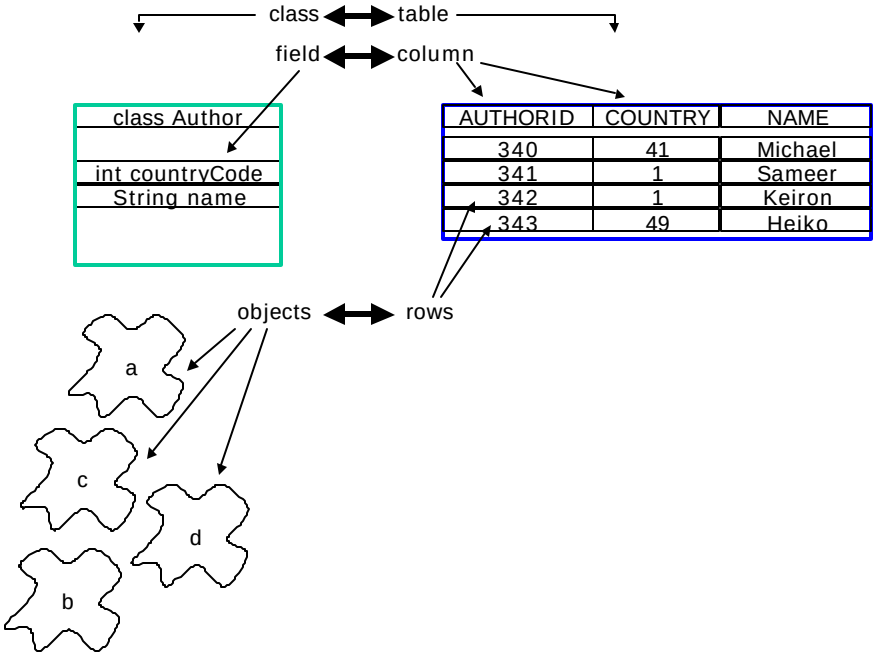

Figure 1: The object vs. the relational view of the world

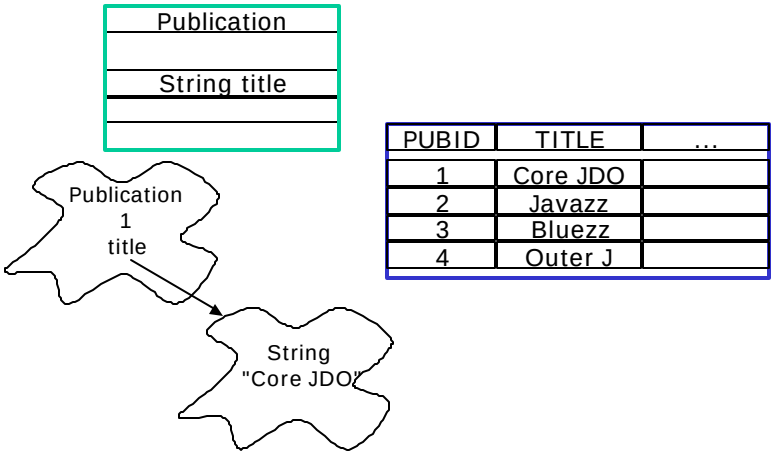

Figure 2: A Java String is really a reference to an independent object, but is generally mapped to a column of a table, not a row in a STRING table

One talks about O/R-mapping if tables and columns of a database are connected with classes and fields of an object-oriented language, so that an instance of a class can be used to access or modify data of one or more rows/columns of that database.

Figure 1 shows a Java class `Author` that has a `countryCode` and `name` field. This information can be put into an `AUTHORS` table that has corresponding `COUNTRY` and `NAME` columns. Instances of the `Author` class are plain Java objects, that contain the `countryCode` and `name` data during runtime. The instance data corresponds to table rows, which are accessed by SQL queries and insert, update or delete operations. Starting with this basic apparently simple mapping scheme the details can get more complex rapidly:

1. In each of both worlds there are different type systems.
2. Java supports inheritance, most database systems don't.
3. Ambiguities exist in mapping references to relations.

Before even delving into some more details, another issue deserves mentioning: Simple class-to-table and attribute-to-field name mapping needs to be addressed. Class names can be long, are case sensitive, and include a hierarchical package name; attribute names can also be long. SQL table and column labels however are often of (very) limited size. Automatic name mapping schemas or manual specification of table and column names are imaginable, and often both provided by JDO implementations and schema generators.

2.3.2. String and Date and other type mappings

The Java type system is different to SQL's type system mainly because basic types like Java short, int, float, double do not directly map to exact SQL types and vice versa.

Even simple types like Java String and Date have no exact equivalent in the database world since they are classes, and in priciple classes are mapped to tables. In figure 2, the Publication class has a title field that refers a String object at runtime, but in a database columns can be declared to store character data.

Using date types is even worse: In Java the Date class counts milliseconds since January, 1st 1970, and stores them internally in a 64-bit long value. In SQL there are many different flavours of date, time, or combinations of them available as column types.

Other mapping issues could arise if the out-of-the-box provided object to table mapping is not desired purely for efficiency reasons in the relational model; it is e.g. often the case to use object references to model a Java "enum" pattern that would generally be modelled as a simple number column. Even simpler, sometimes a String field of a Java class should actually be represented as a small 1-2 character type column with some sort of fixed back and forth mapping of some code, or currency symbol, etc. (Such issues are more likely to arise if existing relational schemas should be used by, but sometimes even desired on new schemas.)

2.3.3. Inheritance mapping

Object models using inheritance can be mapped to relational models by different mapping schemes. The class hierarchy shown in figure 3 will be used throughout the next paragraphs to explain these schemes.

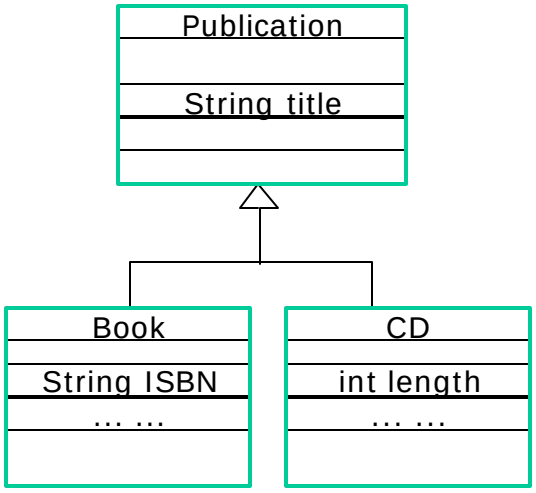

Figure 3: Sample class inheritance

The first scheme maps every field of every class of the class hierarchy into a single, plain table. An additional TYPE column indicates whether a record is a Publication, Book or CD object. Since an instance of Publication can be either a Publication, a Book or a CD, the

table will contain a lot of empty fields. If the instance is a Book, the CD fields are not used and vice versa. Figure 4 shows this table structure.

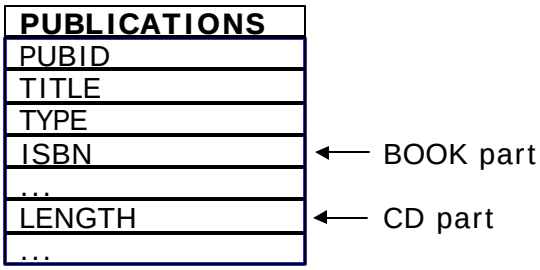

Figure 4: A "flat" Inheritance Mapping with an entire class hierarchy in one table

A second scheme is on the other extreme: It puts the fields of each class in a separate table. A fourth table, BOOKCDPUB defines the relations of the instances. Figure 5 is an example of such a mapping scheme. This scheme should be used if Book and CD instances a rarely expected and a huge number of plain Publication objects exist in the database.

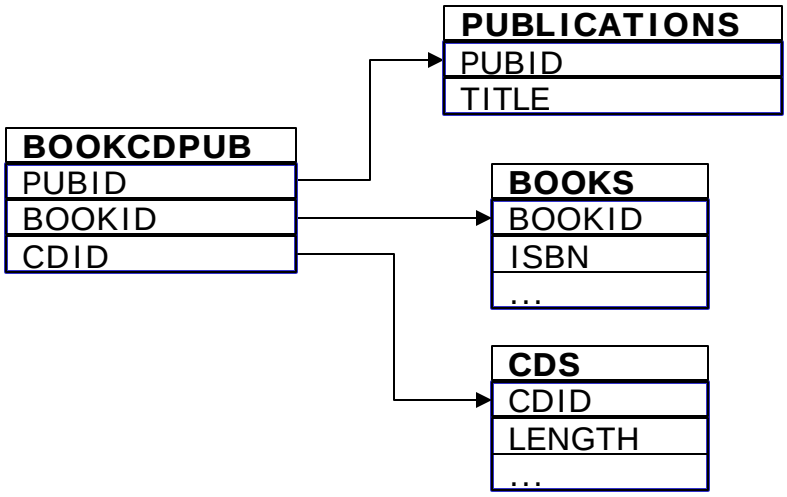

Figure 5: A "fat" inheritance mapping with one table for each class, plus an additional "forward join" table (rarely used by products in this form)

In a third scheme, to save the extra table, if the Publication class is abstract, or no pure Publication objects exist, the Book and CD relation fields BOOKID and CDID can be moved into the Publications table. This scheme, shown in Figure 6, is frequently used in applications. It is

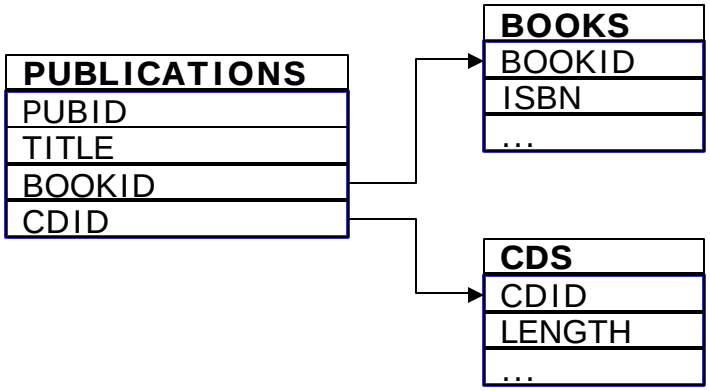

Figure 6: An inheritance mapping with one table for each concrete class, and "forward join foreign keys" from superclass to all possible subclasses

a typical model with key/foreign-key relations, which also illustrates the ambiguity of "is-a" and "has-a" references. One cannot tell from the table declarations, whether Books and CDs are part of Publication data or just referenced by it.

Figure 7 shows a forth mapping scheme that eliminates the extra table by using backward references. Notice that the BOOKID and CDID columns are both primary keys of their respective tables, as well as foreign keys to PUBLICATIONS.PUBID. The insert, update and delete operations for Book and CD objects will be faster and less disk space is used for each object, though queries can become quite complex, for instance if Book objects are searched by title. An often-seen sensible optimization is to provide some sort of TYPE (or CLASS) column in PUBLICATIONS which allows to determine the exact type of a Publication instance, i.e. whether it is "just" a Publication, or a Book or a CD instance, without further queries. Notice how in the previous mapping scheme this was determined by the non-NULL presence of BOOKID or CDID foreign keys in the PUBLICATIONS table.

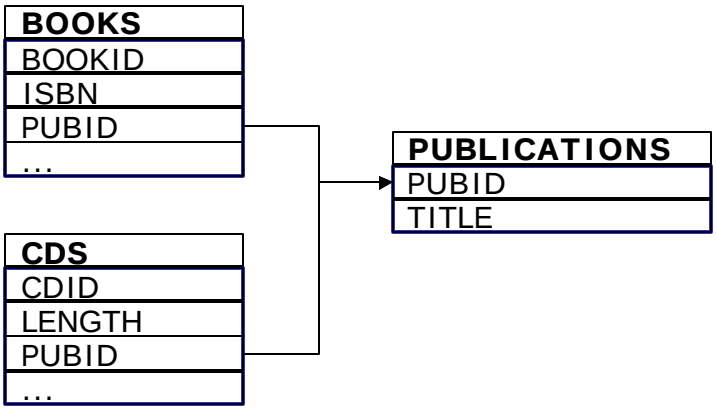

Figure 7: An inheritance mapping with one table for each concrete class, and "backward join foreign key" from each subclass to the superclass

Figure 8 shows yet another possible mapping scheme, where one table per concrete class is used. The definitions of the attributes of the superclass are duplicated in each subclass, but no data is duplicated, as an object is always either a Publication, Book or a CD. If the Publication class is not abstract, a PUBLICATIONS table, with only a PUBID and TITLE column, and storing only instances of real Publication objects, no subclass instances, can be used; if Publication is abstract, then that table is not necessary.

This mapping has very good performance characteristics for most query operations, except for queries on the Publication superclass "with subclasses", which could not be expressed as table join and would lead to two separate queries or a UNION. (It is imaginable, purely to optimize performance for this specific case, to duplicate some columns of records of the subclass tables (BOOKS and CDS) into the superclass PUBLICATIONS table to speed up respective queries; in this case a persistence framework would have to ensure consistency.)

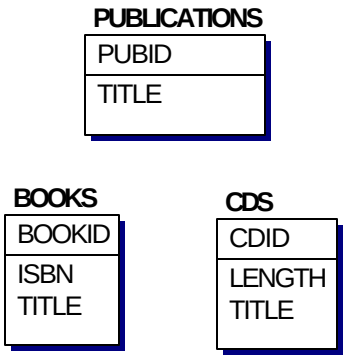

Figure 0-8 An inheritance mapping with separate tables for each concrete class

2.3.4. Security

Other problems when using O/R-mapping result from different access restrictions to data. In Java, public, protected, private and package access defines the rules for field read and write access, as well as method invocation.

Relational database systems have much more flexible access policies, based on user or group security control settings. Such mapping problems may occur, if a column of a table must not be read by a user, but the class does contain a field for that column. Security issues may influence the way columns are mapped to fields and classes to get a NullPointerException for illegal field access.

2.3.5. Query language translation

Finding relational records is usually done by expressing a search in SQL, the Structured Query Language. However, SQL is not well suited to express filtering expressions on collections of objects.

Other chapters of this book (e.g. JDBC & JDOQL chapters) provide some more background on this issue, for the purpose of this point, it's sufficient to note that most object-relational (O/R) mapping tools have a some or the other (String or tree of objects) non-SQL query facility that is then translated into SQL before being sent to the database for execution.

While again simple cases are easy to translate, the more advanced scenarios can rapidly get fairly interesting exercises, i.e. finding which tables to joins, how to perform sorting on attributes of referenced objects (if the object query language allows this, and

JDOQL for example does), when to use Outer Joins, if available, how to express and if allowed how to map SQL functions etc.

2.3.6. Referential Integrity, Deletion etc.

Pure Java objects exist as long as at least one other object references them (Garbage Collection), and the days of C and other low-level programming languages with the problem called "dangling pointers" (references to non-existing objects respectively previously deallocated memory) that lead to screaming developers are long gone when using Java.

However, persistent objects can usually be deleted explicitly. Relational databases have for long had their own mechanism (Referential Integrity constraints) to prevent "dangling records", i.e. foreign key references to non-existent primary keys.

Again, marrying these two different views is an issue that a mapping layer needs to address, be it by explicitly supporting relational referential integrity constraints and performing correct statement ordering if required, maybe supporting cascaded delete options, or by explicitly not supporting relational referential integrity.

Transparent Persistence in O/R-Mapping

Once the Java object model is mapped to columns and tables, an O/R-mapping implementation must be able to read and write objects by some means. The basic operations on databases are query, insert, update and delete. The fundamental idea of transparent persistence is to hide away database access and operations like insert, update and delete, as explained in the previous chapter. A further objective is to minimize database access, like reading from a cache and updating only modified objects or even fields.

Tools are provided by O/R-mapping solutions that create the required code behind field access methods to transparently execute appropriate SQL statements. Though it is possible to implement all the methods by hand, it can be very erroneous. Especially when the model changes from one application version to another, or when the model becomes more complex, many project teams spend more time implementing JDBC and SQL mapping code than implementing application logic. An in-depth view is taken at this problem in chapter #### (JDO and JDBC comparison).

A very brief example of what happens is given below. Assuming there is already an instance of a Book in memory, the following list shows how an O/R implementation retrieves an Author object. The Java code looks like:

```
Book book = ... already in memory...;
Author author = book.getAuthor();
```

The O/R-mapping implementation might perform something similar to these steps:

- Take the book's AUTHORID
- Create an Author instance

- Create a JDBC statement like
  `SELECT * FROM AUTHORS WHERE AUTHORID = id`
- Copy necessary fields of the result record into the author instance. This step includes more or less complex attribute to field mapping due to the different type systems.
- Return the author instance.

2.3.7. Identities

In the previous paragraphs just a single object or class has been part of examination. How does O/R-mapping work with various objects? In the context of an object graph, identities link between objects in memory (or Java references) and rows/tables of a database. These identities are related often to primary key columns in tables though this is not a requirement.

It is essential that a one-to-one association exists between objects in memory and records in a database; else it becomes impossible to identify the appropriate rows for query, update and delete operations. Nobody would expect that a call to book.getAuthor() returns different objects in two adjacent calls. Therefore the pseudo code from the above example has to be extended to satisfy the constraint:

- Take the book's AUTHORID
- Check, if an instance corresponding to AUTHORID already exists
  If yes, return the instance
- Create an Author instance
- Create a JDBC statement like
  `SELECT * FROM AUTHORS WHERE AUTHORID = id`
- Copy necessary fields of the result record into the author instance
- Return the author instance.

JDO supports three different flavors of identities: application identity, data store identity and non-durable identity. Application identity is appropriate for key values that are created outside of the application, like ISBN. Database identity can be used for tables that have no native key, like PUBID in the above examples. Values of this identity type are created by the database system or the O/R-mapping implementation. Various strategies for this purpose exist, and relational databases often have a built-in mechanism (SQL sequences) that could be used. Non-durable identity is used for objects that are not referred by other persistent objects in the database, like bulk data or simple lists.

2.4 ROLLING YOUR OWN PERSISTENCE MAPPING LAYER

After having explored some object-relational specific mapping issues above, the following section will look into some topics which are not strictly O/R related, but apply to any persistence framework that maps some native datastore, be it relational, an object database, XML repositories, or any other imaginable data store.

Same as the above O/R questions, such issues are addressed by existing products. Their understanding however, or at least awareness of, helps developing with such tools. Furthermore, such topics would have to be addressed if you were interested in "rolling" your own persistence layer to implement a JDO-like transparent persistence framework with good performance characteristics. This section will conclude with a short "build vs. buy" discussion.

2.4.1. Caching

Now that objects can be instantiated on-demand and unique references are returned for unique data in the database, it is a small step to realize further caching. Why should a call to getAuthor in the above example do anything at all, if the author object was previously already retrieved? Just the firstly returned Java reference can be returned again. The following pseudo code illustrates the idea:

- Check, if the book instance already contains valid data
  If yes, return the Author reference
- Take the book's BOOKID
- Create a JDBC statement like
  `SELECT * FROM BOOKS WHERE BOOKID = id`
- Copy necessary fields of the result record into the book instance
- Create an Author instance
- Copy the AUTHORID from the result record into the Author instance
- Mark the Author instance, so that it is loaded when a get() method is called
- Return the author instance.

In this case the on-demand loading of objects is split into instantiation of invalid objects that are just placeholders for corresponding database identities, and the actual data retrieval from the database.

Whenever objects are modified, the actual update operation can be delayed until all work related to the object graph is done. This would save unnecessary update operations for the same object or dependent objects. Diverse solutions to find out about changes to objects of a graph of objects exist and are realized in O/R-mapping tools. One can copy objects and

compare each original object with the application object through Java reflection. Set and get methods for fields may be created by a tool to track modifications and load data on demand.

All these solutions have in common that the application has the notion of some point in time when all modifications have to be copied out to the data store. Obviously the same is true for reading: at some point in time the cached objects need to be thrown away (cache eviction) and data has to be reloaded. Various strategies exist, usually linked to some time or memory limit exhaustion; and often implemented in Java by using SoftReferences and WeakReferences and the like.

2.4.2. Transactional Database Access and Transactional Objects

The above mentioned cache consistency is directly connected to transactional database access. Though many database applications are implemented without a proper transactional design, developers trust a database system and rely on ACID[1] properties. A transactional system fulfilling these properties guarantees that it can commit some pieces of work in a single step. Either everything or nothing is done, independent of other parts of the system or separated from them. All work is committed persistently so that the system can fail afterwards without data loss.

It seems that these requirements are hardly ever needed; however it makes sense to link the in-memory object cache to a transactional workflow. Otherwise objects have to be read and written by load and store methods, which lack transparent access mentioned earlier.

A typical workflow with transactional access looks like the following:

- Start a new transaction
- Lookup a first instance as a root of the object graph by using a query or a provided application identity
- Navigate through the object graph
- Modify or delete objects
- Add new objects
- Commit or abort the transaction

Beyond the "simple" usage of underlying datastore transactions as just described, some persistence layers, and JDO very much certainly does as we'll see later, add a real "Transactional Objects" mechanism on top, as pure Java objects clearly are not transactional.

For example, if a transaction is started through the persistence layer (and notice that the Java language as such, like most current programming languages, does not have a notion, e.g. a keyword for, transactions) then some attributes of an object are set, then that

1 ACID is an acronym for "Atomicity, Consistency, Isolation, Durability"

transaction is e.g. aborted, a mechanism like outlined above could certainly correctly inform a transactional database.

The Java object itself however, would keep the old, now invalid, values. In a persistence framework with true transactional object support, which JDO for example is, if so desired and configured, the values of the attributes of the object itself would "rollback" as well. Implementing such behavior is possible by e.g. re-reading the object's content or, much better performing, through integration with a transactionally safe global object cache.

2.4.3. Locking

Another issue, somewhat related to correct Transaction handling in persistence layers, is "Locking". What happens in the case of concurrent both read and write access to the same object? Different strategies exist as we'll examine in the Transaction chapter, but whatever the details, a persistence layer again needs to make an extra effort for this purpose.

For example, to support optimistic locking on a relational database, an extra timestamp column or incremental numeric counter column on each table is often introduced, kept up to date, and checked. For pessimistic locking, the "... FOR UPDATE" SQL syntax (non-standard) may have to be used. Such underlying locking implementations have to be implemented transparent to the application logic and translated to the respective API.

2.4.4. Arrays, Sets, Lists and Maps

The Java language and more so the standard (java.util) Collection library have various ways of modeling associations between objects, from simple Arrays, to Sets, to Lists and finally Maps.

Different data stores have different and sometimes limited direct support for such models. Relational data stores for example out-of-the-box (with the ubiquitous "join table in the middle") really model only what a correct Java model would express as a Set, while in XML-to-object mapping speaking a collection of objects is always ordered, thus what Java would model with a List type.

Should different such associative models be permitted, work exactly like their Java interface contract requests, and perform well for large data and associations, including query operations, then extra mapping, conversion and effort needs to be undertaken by a mapping layer.

2.4.5. Performance and Efficiency

Questions of Performance and Efficiency arise with many aspects of the implementation of a persistence layer, for example:

- How are attributes of the persistent class read and written? Access to fields of objects through reflection is very slow. JDO implementations

generally for example do not actually use Java Reflection. Other persistence layers have different approaches.

- How are large query results (collection of objects) handled? Is everything loaded at once, or using some sort of Cursor or Iterator architecture? Is pre-fetching a certain number of initial records possible? (Like SQL's "OPTIMZE FOR x ROWS" or Oracle's "/* FIRST_ROWS */" syntax)
- How are large associations between e.g. two objects handled? Is partial object reading with basic lazy loading of referenced objects and "non-default fetch group" attributes supported? Is a Set or List always fully loaded into memory, or are there optimized framework specific implementations of such interfaces, or are techniques such as Java Proxies implemented? Are such purely performance enhancing optimizations transparent to an application of a framework, or do some sort of ValueHolder-type classes have to be used explicitly?
- Is there any optimization before interacting with the underlying database; i.e. are "redundant" operations within one transaction (e.g. multiple sets on same attributes) recognized and simplified? Are protocol specific features utilized, e.g. for O/R mapping through JDBC, reusing of prepared statements, batch statements, etc.

2.4.6. Build Vs. Buy of a Persistence Framework

The previous pages have outlined the issues that a Persistence Framework in general and an O/R mapping based one in particular need to address. The issues presented are not exhaustive, and questions such as complex object life cycle definition and implementation used to model some of the behavior outlined has not even been discussed. So in short, while such a framework certainly could be built in-house, it is worth a detailed "build versus buy" cost and effort analysis.

This book can not attempt to provide estimate figures, but merely outlined what is involved in case of a "build" decision. People who have tried "build instead buy" usually argue for a "buy instead build" afterwards...

Alternatively, a much simpler and less generic framework may be envisioned for "build" in rare situations. As for any major project, clearly defining the scope of a framework to build is essential. Areas that possibly provide room for simplification include:

- Not real transparent orthogonal persistence, "no persistence by reachability", but simple explicit "save" or "update" methods (instead makePersistent) on each modified persistent object.
- Not real transactions and transactional objects. No locking. Not a great architecture, but many simple JDBC applications do work like that.

- Limited query capability, no generic query language, and thus no translation. Only special application-specific cases. Alternatively, usage of SQL as query language, with no support for queries via object navigation etc.
- No support for inheritance
- No caching
- Etc.

2.5 CONCLUSION

O/R-mapping should be used when existing database applications have to run together with newly developed applications, for instance a mature procurement system that needs to be accessed via the Internet. Existing data analysis tools like OLAP-based data mining tools or reporting solutions can be a reason to use O/R-mapping as well. To become independent of database vendors may be another advantage of O/R-mapping tools. Though it is doubtful that an application runs out of the box on a different database system with expected performance, a mapping layer helps to port such applications in fractions of time. The “JDO and JDBC” chapter in this book examines these issues in some more detail.

If the application uses a deeply hierarchical object model, like XML document-oriented or CAD applications, a relational database system can be too less performing. Navigational access may result in a number of queries, which can be difficult to tune. In such case object-oriented or XML-based databases are a better choice. In the embedded domain even a database system may be too large because of restricted memory and code space. Nevertheless there are small, file-based solutions that can be used via the JDO API.

While the previous section has focused on O/R-based transparent persistence, many of the issues discussed remain the same for non-relational data stores. For example, even if data is persisted in an object-database like or native XML store, the essential questions of transactional objects, caches and query translations from one language into another remain.

If the object model is fairly simple and stable, using O/R mapping tools may be 'overkill' compared to the effort of an own, application-specific persistence implementation. The decision should also depend on resource consumption, number of parallel accesses, data consistency requirements and the need for query capabilities. A really bad idea is to design another mapping layer on top of JDO, to implement the application independent of various persistence frameworks like ODMG, JDO and other, vendor dependent solutions. The fundamental purpose of JDO, transparent persistence, will be lost in such a case.